Examen Programación 1º
DAM/DAW Semipresencial
19 febrero 2025
2ª Evaluación

I.E.S. Doctor Balmis

GENERALITAT VALENCIANA
Conselleria de Educación, Cultura y Deporte

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Nombre	

Instrucciones para la realización de la prueba:

- Duración de la prueba **2 h 40 m** desde el comienzo de la misma.
- Lee atentamente el enunciado y no formules tus preguntas en voz alta, levanta la mano y espera a que el profesor te diga que tienes que hacer.
- Si te atascas en un apartado déjalo y pasa al siguiente. Cada apartado tiene su puntuación, independientemente del funcionamiento del programa final y el orden propuesto en el enunciado.
- Penalizará con **0,1 décimas** en cada uno de los siguientes casos:
  - Llaves mal puestas e instrucciones mal tabuladas dentro de un bloque.
  - No seguir las reglas de nomenclatura de C#.
  - Identificador de variable o método que a juicio del profesor no esté bien puesto porque no sigue las reglas descritas en classe.
  - No definir cada tipo en un fuente del mismo nombre.
- **No se puede usar ningún tipo de dispositivo USB, ni código de ejercicios o ejemplos realizados en clase. Solo los pdf de apuntes que se te proporcionarán para la ocasión**.
- En caso de pillar a algún alumno copiando o usando material no autorizado se le retirará el examen y su calificación en la evaluación será un suspenso.

Pregunta 1 (4,5 puntos)

El Restaurante "El Bug del Código"

Eres un desarrollador contratado para crear un sistema que genere menús para "El Bug del Código". El chef, con su particular sentido del humor, te pide tres opciones: "***Menú Sorpresa***", "***Menú sin Gluten ni Ajo***" y "***Menú Reducido***".

En los recursos para el examen, en el workspace del ej1, se te provee con las clases `Plato`, `Entrante`, `Principal`, `Postre`, `Bebida`, `ValorNutricional`, el **enum** `Alergeno` y la clase estática `BancoDePlatos` que contiene arrays de platos predefinidos. Dedica unos minutos a analizar la estructura de las clases y los métodos que contienen.

1. **Crea la clase abstracta** `Menu`:
   - Debe tener un campo **array de platos** con un **método** setter protected y un getter privado.
   - Método abstracto `TipoMenu()` que usaremos en el `ToString()` y que me devuelva una cadena con el nombre de los posibles tipos de menú.
   - Define los métodos `Precio()` que calcula el precio total del menú, `ValorNutricional()` que calcula la suma del valor nutricional de los platos y `Alergenos()` que devuelve la combinación de todos los alergenos presentes en el menú.
   - Invalida el método `ToString()` para mostrar el menú de forma legible, incluyendo el nombre de cada plato, sus ingredientes, precio y alérgenos, así como el precio total, el valor nutricional total y los alérgenos del menú completo.
2. **Crea la clase** `MenuSorpresa`: Hereda de `Menu` y genera un menú aleatorio con un entrante, un principal, un postre y una bebida obtenidos de los platos predefinidos en `BancoDePlatos`.
3. **Crea la clase** `MenuSinGlutenNiAjo`: Hereda de `Menu` y genera un menú que **no contenga el alérgeno *'gluten'***, **ni el ingrediente *'ajo'*** en ninguno de sus platos. Si no es posible generar un menú con estas restricciones, lanza una excepción personalizada del tipo `MenuException` con el mensaje "*No se puede generar un menú sin gluten ni ajo*". Para ello, puedes añadir **el primer plato** que no contenga gluten ni ajo de cada tipo de plato. Si no encuentra.
4. **Crea la clase** `MenuReducido`: Hereda de `Menu` y genera **un menú reducido** con un plato único que se recibirá como parámetro en un **único** constructor de la clase. Además, tendrá un postre y una bebida aleatorios.
   El plato único podrá ser solo un **entrante** o un **principal**, pero no un postre ni una bebida. Si el plato no es un entrante ni un principal, lanza una excepción personalizada del tipo `MenuException` con el mensaje "*El plato para crear un menú reducido debe ser un entrante o un principal*".
5. **Completa** el programa principal que se te proporciona para que:
   - Crear las instancias del menú que se proponen en las opciones dentro del switch de expresión.
   - Controlar las excepciones de tipo `MenuException` que se produzcan dentro del while y mostrar el mensaje de error correspondiente y recuperando la ejecución normal del bucle.

Ejemplo de Salida y criterios de evaluación en la siguiente página.

```
1. Menú sorpresa
2. Menú sin gluten ni ajo
3. Menú reducido con principal "Paella Valenciana"
4. Menú reducido con entrante "Ensalada César"
5. Salir
Seleccione una opción: 4

Menu Reducido

Entrante
Nombre: Ensalada César
Ingredientes: Lechuga, pollo, picatostes, salsa César
Precio: 8,5
Alergenos: Gluten, Lacteos
Postre
Nombre: Flan de huevo
Ingredientes: Huevos, leche, azúcar, caramelo
Precio: 4
Alergenos: Lacteos, Huevos
Bebida
Nombre: Agua mineral
Ingredientes: Agua
Precio: 1,5
Alergenos: Ninguno

Precio total: 14
Alérgenos: Gluten, Lacteos, Huevos
Valor nutricional: Calorías 650, Grasas 40, Azúcares 35, Proteínas 25
```

Criterios de calificación:

- (✅ **0,25 pts**) Definición de `Menu`, campos, accesores/mutadores y método abstracto.
- (✅ **0,25 pts**) Implementación del método `Precio`.
- (✅ **0,5 pts**) Implementación del método `ValorNutricional`.
- (✅ **0,5 pts**) Implementación del método `Alergenos`.
- (✅ **0,25 pts**) Implementación del método `ToString`.
- (✅ **0,25 pts**) Implementación de la clase `MenuSorpresa`.
- (✅ **1 pts**) Implementación de la clase `MenuSinGlutenNiAjo`.
- (✅ **0,75 pts**) Implementación de la clase `MenuReducido`.
- (✅ **0,25 pts**) Completa el programa principal según las especificaciones.
- (✅ **0,5 pts**) Compila y ejecuta correctamente.

Pregunta 2 (5,5 puntos)

¡Alerta Extraterrestre! 👽

Año 3050 la tierra no para de recibir amenazas extraterrestres y la **LNC** (Liga de Naciones Cibernéticas) ha decidido crear un sistema de alerta temprana para clasificar los mensajes recibidos según su nivel de amenaza.

En los recursos para el examen, en el workspace del ej1, se te provee el archivo `mensajes_extraterrestres.log` con los mensajes interceptados por la LNC. Este archivo contiene entradas de texto con el siguiente formato:

```
Fecha - Hora - Planeta - Especie - Nivel de amenaza - Mensaje
```

Donde:

- `Fecha`: Fecha en formato `dd/MM/yyyy`.
- `Hora`: Hora en formato `HH:mm:ss`.
- `Planeta`: Nombre del planeta de origen.
- `Especie`: Nombre de la especie extraterrestre.
- `Nivel de amenaza`: Puede ser "*Bajo*", "*Medio*" o "*Alto*".
- `Mensaje`: Un mensaje en lenguaje alienígena (una secuencia de caracteres aleatorios).

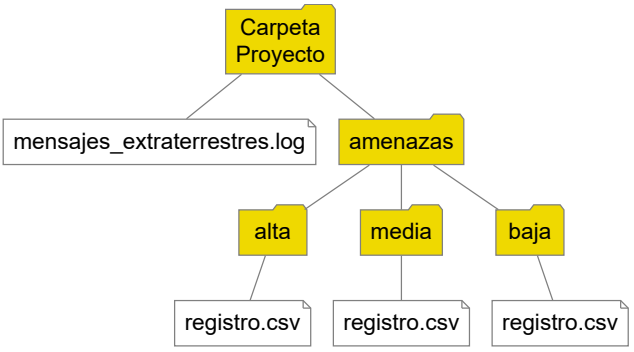
Ejemplo de mensajes:

```
22/01/3050 - 12:00:00 - Kepler-186f - Floofians - Bajo - !@#%^&*()_+=-`
25/01/3050 - 12:05:15 - Proxima Centauri b - Reptilianos - Alto - ~!@#$%^&*()_+=-`
30/02/3050 - 11:10:00 - Trappist-1e - Gleepglorps - Medio - !@#$%^&*( )_+=-`
```

Descripción general del funcionamiento:

Leer el archivo `mensajes_extraterrestres.log`.

En el directorio de ejecución del programa, crea una carpeta llamada `amenazas` y dentro de ella tres carpetas llamadas `alta`, `media` y `baja` según el tipo de alertas que se reciban.



Clasificar cada mensaje en el directorio correspondiente según su nivel de amenaza en un archivo denominado `registro.csv`. Este archivo contendrá una línea por cada mensaje, separando los datos por comas con el siguiente formato:

```
Fecha,Hora,Planeta,Especie,Mensaje                <- Cabecera
dd/MM/yyyy,HH:mm:ss,Planeta,Especie,"Mensaje"     <- Registro
```

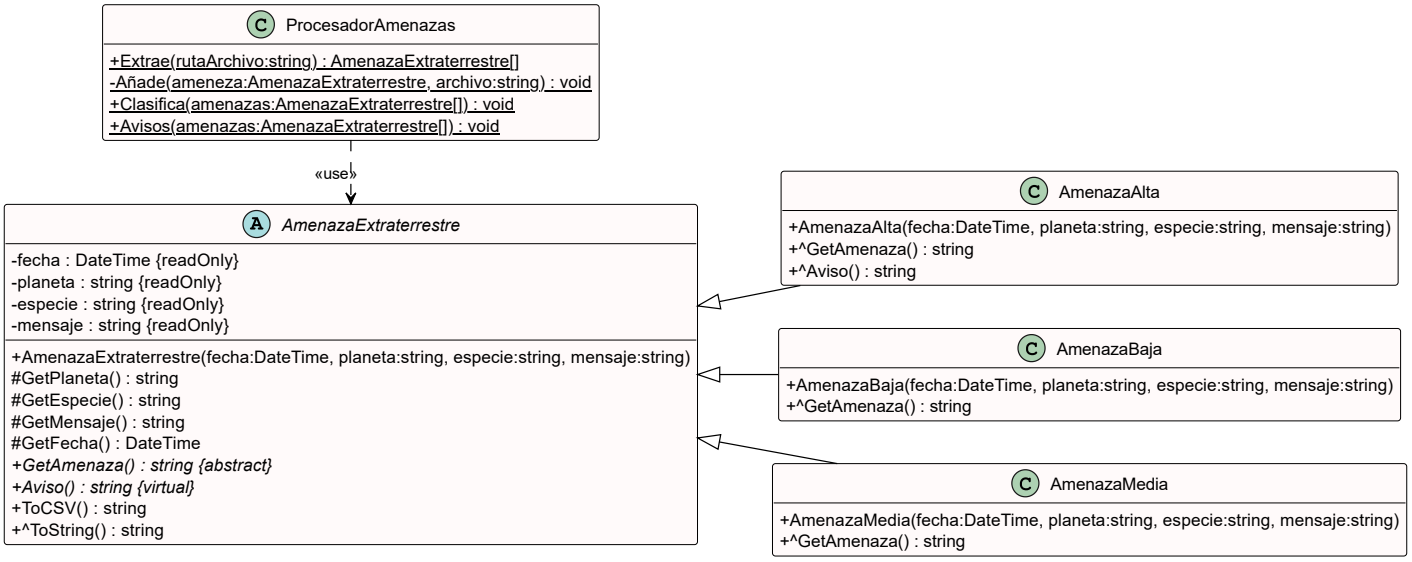
Por ejemplo, para los mensajes de nivel "*Alto*" generará ...

```
Fecha,Hora,Planeta,Especie,Mensaje
25/10/2021,12:05:15,Proxima Centauri b,Reptilianos,"~!@#$%^&*()_+=-`"
01/01/2022,8:09:10,Gliese 581g,Zorgonianos,"&*%^#$#@#$%^&"
...
```

> **Importante**
>
> Los archivos de registro solo se crearán si encontramos alguna alerta de ese tipo y si no existen previamente. Si ya existen, se añadirán los nuevos mensajes al final del archivo.

Especificaciones de implementación:

Para solucionar el ejercicio se propone el siguiente diseño del diagrama de clases UML de abajo que deberás seguir:



Donde:

1. Tenemos una **clase abstracta** `AmenazaExtraterrestre` con los campos `fecha`, `planeta`, `especie` y `mensaje` de solo lectura. Donde...
   - El método **virtual** `Aviso()` una cadena con un texto donde se indique la especie y el planeta detectados como por ejemplo: "*Floofians detectados en Kepler-186f*".
   - EL método `ToCSV()` devuelve como cadena un registro del archivo CSV con el formato indicado anteriormente. **Fíjate que el mensaje debe ir entre comillas dobles**.
   - El resto de la implementación debes deducirla del UML representado la accesibilidad descrita.
2. Las concreciones de amenaza descritas en el UML donde `GetAmenaza()` devolverá un texto con el tipo de amenaza "*Alta*", "*Media*" o "*Baja*" según corresponda.
3. Invalidaremos el método `Aviso()` en `AmenazaAlta` para que anteponga el texto "*ATENCION !!!*" al mensaje generado en la clase base. Solo si la especie son los "*Reptilianos*" o los "*Insectoides*".
4. `ProcesadorAmenazas` es una clase estática que contiene los métodos:
   - `Extrae` Que procesará el archivo de log y devolverá un array de `AmenazaExtraterrestre` procesando cada línea del archivo con la **expresión regular que se te proporciona en los recursos**.
     Para crear una fecha a partir de una cadena puedes usar el método `DateTime.Parse($"{fecha} {hora}");`.
   - `Añade` Es un método privado qie usaremos en `Clasifica` y que se encargará de añadir un registro de amaneza al archivo CSV que recibe como parámetro, anteponiendo la cabecera con los nombre de las columnas si el archivo no existe.
   - `Clasifica`: Usará el método `Añade` y se encargará de clasificar las amenazas en los directorios y archivos de registro correspondientes según el nivel de amenaza. Controlando las excepciones `IOException` y `UnauthorizedAccessException` que se puedan producir y recuperándose de las mismas.
   - `Avisos`: Mostrará los avisos generados por cada amenaza en la consola.

Se te proporciona el los recursos el programa principal que utilizará dichos métodos.

> **Criterios de calificación:**
>
> - (✅ **1 pts**) Implementación de la clase `AmenazaExtraterrestre` con los campos y métodos descritos.
> - (✅ **0,25 pts**) Implementación las subclases de `AmenazaExtraterrestre` con los métodos descritos excepto la invalidación de `Aviso()`.
> - (✅ **0,75 pts**) Invalidación de aviso en `AmanazaAlta` según las especificaciones.
> - (✅ **1 pts**) Implementación del método `Extrae` de la clase `ProcesadorAmenazas` utilizando la ER propuesta.
> - (✅ **0,5 pts**) Implementación del método `Añade` de la clase `ProcesadorAmenazas`.
> - (✅ **1 pts**) Implementación del método `Clasifica` de la clase `ProcesadorAmenazas`.
> - (✅ **0,5 pts**) Implementación del método `Avisos` de la clase `ProcesadorAmenazas`.
> - (✅ **0,5 pts**) El programa compila y ejecuta correctamente de acuerdo a la especificación.